Autonomous Drone Swarm Simulation

Research Guide: Areas, Theories and Technologies, and the Path from Simulation to Real-World Systems

What this document is
A study companion to the project: it lists the research areas, topics and sub-topics behind it, explains the key theories with the equations exactly as implemented, summarises the technologies and results, records the lessons learned, and lays out how the work transfers from simulation to real drones and real-world problems, with references.

Thilina Samarasinghe

November 2026

github.com/Thilina-Samarasinghe/autonomous-drone-swarm

Contents

1. Project overview **3**

2. Research map: areas, topics and sub-topics **4**

3. Core theories and how they were applied **5**

4. Technologies and tools **9**

5. System architecture **9**

6. Key results **11**

7. Lessons learned **14**

8. From simulation to the real world **15**

8.1 The reality gap 15

8.2 A step-by-step path to real drones 15

8.3 Mapping each module to its real-world counterpart 15

8.4 Real-world applications 16

8.5 Safety, legal and ethical considerations 16

9. Future research directions **17**

Glossary **17**

10. References **18**

1. Project overview

The project is a 2D simulator of a swarm of 20–50 autonomous drones written in Python. **No central controller exists.** Each drone decides for itself from what it can sense (neighbours within 15 m, a 36-beam lidar), what it has mapped itself, and what teammates tell it over a lossy, range-limited radio. Cooperative behaviour (flocking, formations, shared maps, search and rescue) *emerges* from these local decisions. That is the defining property of swarm intelligence and the main research theme of the project.

It was built in ten phases, each adding one capability and each verified by automated tests (189 in total) and by benchmarks repeated over several random seeds. The results below are measured, not assumed.

Phase	Capability	Headline result
0–1	Simulation core, boids flocking	50 drones flock with 0 collisions; local alignment 0.97
2	Lidar, potential fields, ORCA	ORCA: 0 collisions and 0 obstacle contacts in all 15 runs
3	Radio, occupancy maps, multi-hop sharing	Each drone knows 85% of the arena after 30 s vs 17% alone
4	Path planning (A*, cost field)	No planning: 0% escape the maze; planning: 100%; shared maps 29% faster
5	Formation flying	V / line / box / circle; through gaps and a maze with nobody left behind
6	Search and rescue	8/8 victims found in every run (random search: 3.7/8)
7	Terrain and threats, PSO routes	0 s in lethal zones; 4–5× fewer close encounters than grid routes
8	Multi-agent reinforcement learning	RL + ORCA ~25% faster; RL alone not as safe as hand-written control
9	Fault injection and robustness	0 collisions in 68 stress runs; 8/8 victims with 90% packet loss

Design decisions. 2D world and point-mass dynamics (every algorithm transfers to 3D); pure Python so it runs on a CPU-only laptop (Intel i5-8365U, no GPU); one controller interface so hand-written and learned controllers are interchangeable and compared fairly; a seeded, fixed-time-step engine so every experiment is reproducible.

2. Research map: areas, topics and sub-topics

The project sits where several research fields meet. The table lists each area, the topics studied, and the concrete sub-topics that were implemented (I) or studied for comparison (S).

Research area	Topics	Sub-topics (I = implemented, S = studied)
A. Multi-agent systems and swarm intelligence	Emergence, decentralisation, local interaction rules, collective behaviour	Boids (I); order parameter / polarisation (I); consensus and flocking theory (S); stigmergy (S); scalability with swarm size (I)
B. Motion control and collision avoidance	Kinematic agents, steering behaviours, reactive avoidance, provable safety	Point-mass dynamics with limits (I); prioritised acceleration (I); artificial potential fields (I); velocity obstacles, RVO, ORCA (I); 2D linear programming (I); control barrier functions (S)
C. Perception and mapping	Range sensing, probabilistic mapping, sensor uncertainty	Lidar ray casting (I); occupancy grids with log-odds (I); ground-truth evaluation (I); sensor noise and dropout (I); SLAM (S)
D. Networked systems and communication	Wireless ad-hoc networks, information spreading, consistency	Range-limited broadcast, loss, latency, bandwidth (I); beacon tables (I); gossip / epidemic protocols (I); anti-entropy (I); multi-hop relay (I); network connectivity (I)
E. Path planning and graph search	Shortest paths, heuristics, replanning in unknown space	Dijkstra (I); A* with octile heuristic (I); backward cost-to-go field (I); optimistic planning (I); cost maps and inflation (I); string pulling (I); D* Lite, Theta* (S)
F. Formation control	Leader-follower, virtual structure, assignment	Slot tracking with feed-forward (I); Hungarian assignment (I); breadcrumb trail following (I); passage-width estimation (I); leader re-election (I); consensus-based formation (S)
G. Optimisation and metaheuristics	Population-based search, constrained optimisation	Particle Swarm Optimisation (I); inertia-weight schedule (I); hybrid PSO seeded by graph search (I); penalty methods (I); prioritised multi-agent planning (I); Douglas–Peucker (I)
H. Cooperative missions and task allocation	Exploration, coverage, search theory, task assignment, energy	Frontier exploration (I); dynamic Voronoi partitioning (I); consensus auctions / CBBA (I, simplified); battery-aware return (I); signal source seeking (I)
I. Machine learning and multi-agent RL	MDPs, policy gradients, centralised training with decentralised execution	PPO (I); GAE (I); MAPPO-style centralised critic (I); parameter sharing (I); reward shaping (I); curriculum learning (I); generalisation across swarm sizes (I); sim-to-real transfer (S)
J. Reliability and fault tolerance	Graceful degradation, failure handling	Fault injection (I); leader election (I); robustness to radio loss and sensor noise (I); stuck detection and recovery (I)
K. Software engineering and experimental method	Reproducible research, testing, performance	Unit and end-to-end tests (I); multi-seed benchmarks (I); profiling and vectorisation (I); configuration management (I); visualisation (I)

3. Core theories and how they were applied

For each theory: the idea, the equation as it is implemented in the code, where it lives, and what the project taught about it. Vectors are 2D; dt = 0.05 s.

3.1 Point-mass dynamics and steering behaviours

Each drone is a point with position **p** and velocity **v**, limited to $|v| \le 8$ m/s and $|a| \le 6$ m/s². A controller only chooses an acceleration; semi-implicit Euler integration advances the state. Reynolds' steering idea: steering = desired velocity − current velocity.

$$\mathbf{v} \leftarrow \text{clamp}(\mathbf{v} + \text{clamp}(\mathbf{a}, a_{max})\cdot dt, v_{max}); \ \mathbf{p} \leftarrow \mathbf{p} + \mathbf{v}\cdot dt$$

$$\mathbf{a}_{steer} = \text{clamp}(v_{des}\cdot\hat{\mathbf{u}} - \mathbf{v}, a_{max})$$

In the project: `swarm/sim/dynamics.py`, `swarm/control/boids.py`

What we learned: Limits matter: a drone that is told to go somewhere it physically cannot brake for will collide, whatever the algorithm says (Phase 1: drones spawned at full speed collided before they could stop).

3.2 Boids flocking and prioritised acceleration

Three local rules produce a flock: separation (avoid crowding), alignment (match neighbours' heading) and cohesion (move toward the local centre). Reynolds' prioritised acceleration allocation gives safety-critical rules first claim on the limited acceleration budget.

separation: $\Sigma_j (p_i - p_j) / d_{ij}^2$ over neighbours with $d_{ij} < 4$ m (anticipated 0.6 s ahead)

alignment: mean neighbour velocity; cohesion: (centroid − p_i) / $r_{perception}$ · a_{max}

polarisation (order parameter) $\varphi = |(1/N)\ \Sigma\ v_i/|v_i|\ | \in [0, 1]$

In the project: `swarm/control/boids.py`, metric in `swarm/metrics.py`

What we learned: A naive weighted sum let weak rules cancel collision avoidance. Prioritised allocation plus anticipatory separation (reacting to where a neighbour *will* be) removed every collision. A cohesion rule written as “desired − current” acted as a brake and halved the flock's speed.

3.3 Artificial potential fields (APF)

Obstacles create a repulsive force that grows as the drone gets closer (Khatib, 1986). Cheap and smooth, but known to suffer from local minima and to give no safety guarantee.

$F_{rep} = a_{max} \cdot ((d_0 - d) / (d_0 - r))^2 \cdot \mathbf{n}$ for $d < d_0 = 8$ m (plus a tangential term to slide around obstacles)

In the project: obstacle term in `swarm/control/boids.py`

What we learned: APF alone never reached zero obstacle contact (in a crowd it must trade drone separation against obstacle clearance) and, without planning, it traps the swarm at the first wall of a maze.

3.4 Velocity obstacles and ORCA

A velocity obstacle is the set of velocities that lead to a collision within a time horizon τ. ORCA (Optimal Reciprocal Collision Avoidance) turns each neighbour into one half-plane constraint on the drone's own velocity, with each drone taking *half* of the avoidance effort, so no communication is needed. A small 2D linear program picks the allowed velocity closest to the preferred one.

$$ORCA_{A|B} = \{ v : (v - (v_A + \tfrac{1}{2}\mathbf{u})) \cdot \mathbf{n} \geq 0 \}$$

$$v_{new} = \text{argmin}_{v \in \cap ORCA} |v - v_{pref}| \text{ subject to } |v| \leq v_{max}$$

obstacle point at distance d: $\hat{\mathbf{u}} \cdot v \leq (d - r) / \tau_{obs}$ (full effort; obstacles do not move)

In the project: `swarm/control/orca.py` (a port of the RVO2 linear-programming solver)

What we learned: ORCA gave 0 collisions and 0 obstacle contacts in every run, and was the fastest safe method. Wrapped around any other controller (boids, formations, even a learned policy) it acts as a safety layer.

3.5 Occupancy-grid mapping (Bayesian, log-odds)

The world is a grid; each cell stores the probability that it is occupied, kept as log-odds so a Bayesian update becomes an addition. Beams that pass through a cell are evidence of free space; a beam ending in a cell is evidence of an obstacle.

$$l(m \mid z_{1:t}) = l(m \mid z_{1:t-1}) + \log [\, p(m \mid z_t) / (1 - p(m \mid z_t)) \,]$$

implemented: $l \leftarrow \text{clamp}(l + l_{occ} \text{ or } l_{free}, \pm 4)$, $l_{occ} = 0.85$, $l_{free} = -0.4$; known if $|l| \geq 1$

In the project: `swarm/mapping/occupancy.py`

What we learned: Maps were 99.8%+ accurate against ground truth. Clamping keeps cells able to change their mind; merging maps by "more confident wins" is idempotent, so information relayed many times is never double-counted.

3.6 Gossip protocols and anti-entropy

Epidemic (gossip) algorithms spread information by repeated exchanges with neighbours; anti-entropy periodically re-sends known state so that lost messages are eventually repaired (Demers et al., 1987).

merge rules: confirmations → logical OR; best readings → maximum; beacon / bid tables → newest timestamp

each message = newly learned cells + a random sample of known cells (anti-entropy)

In the project: `swarm/comms/network.py`, gossip in `swarm/mission/manager.py`

What we learned: Without anti-entropy a cell whose messages were all lost was never shared again. With it, the swarm kept 81% map coverage per drone even at 50% packet loss, and found all victims at 90% loss.

3.7 Graph search: Dijkstra, A* and cost-to-go fields

A* expands nodes in order of f = g + h, where g is the cost so far and h an admissible (never overestimating) heuristic, which guarantees an optimal path. Running Dijkstra *backwards* from the goal gives the cost-to-go from every cell, the quantity D* Lite maintains incrementally.

$f(n) = g(n) + h(n)$; octile heuristic $h = (\Delta x + \Delta y) + (\sqrt{2} - 2) \cdot \min(\Delta x, \Delta y)$

edge cost = length × ½ ($c_a + c_b$); c = 1 free, +0.3 unknown, +8 inflated, ∞ obstacle

In the project: `swarm/planning/grid.py`, `swarm/planning/planner.py`

What we learned: Optimistic planning (treat unknown space as free, replan when walls are discovered) solved a maze nobody was told about. Tests proved the cost field finds paths exactly as short as A*, about 6× faster.

3.8 The assignment problem (Hungarian algorithm)

Assign n followers to n formation slots minimising total travel distance. Solved exactly in $O(n^3)$.

$\min \Sigma_i C_{i,\sigma(i)}$ over permutations σ, $C_{ij} = |p_i - s_j|$

In the project: `swarm/formation/manager.py` (SciPy *linear_sum_assignment*)

What we learned: Re-solving the assignment after a shape change, or after drones fail, keeps the formation compact with minimal movement; a formation that loses its leader re-forms in seconds.

3.9 Particle Swarm Optimisation (PSO)

A population of candidate solutions (particles) moves through the search space, each pulled toward its own best position and the swarm's best. Used twice: as a *physical* search (drones are the particles, seeking a victim's radio signal) and as an *offline optimiser* (a particle is a whole route plus a departure delay).

$v \leftarrow w \cdot v + c_1 r_1 (p_{best} - x) + c_2 r_2 (g_{best} - x)$; $x \leftarrow x + v$

victim search: $w = 0.6$, $c_1 = 1.2$, $c_2 = 1.6$, plus noise $\propto$ estimated range and restarts after 6 s stall

routes: w decreases 0.9 → 0.4 (Shi & Eberhart, 1998); cost = length + threat + climb + turns + conflicts + delay

In the project: `swarm/mission/manager.py`, `swarm/terrain/route_planner.py`

What we learned: Pure PSO is unreliable with hard constraints (some routes clipped forbidden areas); seeding it with a graph-search route (hybrid) made it reliable. With only three particles and noisy signals, restarts and distance-scaled exploration were what made the victim search work.

3.10 Exploration, coverage and task allocation

Frontier exploration sends robots to the boundary between known-free and unknown space (Yamauchi, 1997). A Voronoi partition gives each robot the frontiers closer to it than to any teammate. Market or consensus auctions (CBBA) allocate tasks: the best bidders win.

drone k takes frontier f only if $|f - p_k| \le \min_{j \neq k} |f - p_j|$ (believed positions from radio)

auction: bid = distance to the best signal reading; keep seeking only while among the 3 lowest fresh bids

In the project: `swarm/mission/manager.py`

What we learned: Exploring the map is not the same as seeing every spot: frontier search found ~7 of 8 victims, the signal-guided PSO found the last ones, and switching the radio off dropped success from 8 to 3.3.

3.11 Reinforcement learning: PPO, GAE and centralised training

Each drone's problem is a partially observable Markov decision process. Proximal Policy Optimisation improves a stochastic policy while clipping how far each update can move it. Generalised Advantage Estimation trades bias against variance. In multi-agent settings, a critic that sees team-wide information during training (centralised training, decentralised execution) stabilises learning.

$L^{CLIP}(\theta) = E[\ \min(\ r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\ 1-\varepsilon,\ 1+\varepsilon)\hat{A}_t\ )\ ]$, $r_t = \pi_\theta(a_t|o_t) \ / \ \pi_{old}(a_t|o_t)$, $\varepsilon = 0.2$

$\hat{A}_t = \Sigma_l\ (\gamma\lambda)^l\ \delta_{t+l}$, $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, $\gamma = 0.99$, $\lambda = 0.95$

In the project: `swarm/rl/` (observation, environment, PPO, deployment as a controller)

What we learned: The details decide success: a shared gradient clip starved the actor; acceleration actions made every task a momentum-control problem; an unobservable penalty (arena walls the lidar could not see) made learning go backwards. Policies trained with 8 drones did not generalise to 20.

3.12 Fault tolerance and leader election

A distributed system must keep working when members fail. Leader election chooses a new coordinator (e.g. the bully algorithm, Garcia-Molina 1982); graceful degradation means performance drops in proportion to the damage rather than collapsing.

```
new leader = min{ id : drone id is alive }; slots re-assigned for the survivors
```

In the project: `swarm/sim/engine.py` (fail), `swarm/formation/manager.py` (set_alive)

What we learned: With half the rescue swarm failing mid-mission, all 8 victims were still found. Noisy lidar (≥ 1 m) was more harmful than missing lidar beams: wrong data is worse than no data.

4. Technologies and tools

Technology	Used for	Why it was chosen
Python 3.12	Everything	Fast to iterate, the standard for robotics research prototypes and ML
NumPy	Vectorised state, physics, sensing, metrics	Batch maths over all drones at once: 25× real time for 50 drones on a laptop CPU
SciPy	cKDTree neighbours; csgraph Dijkstra; linear_sum_assignment; ndimage distance transform	Proven, compiled implementations of the core algorithms
PyTorch (CPU)	PPO actor and critic networks	Standard deep-learning library; runs without a GPU
Pygame-CE	Real-time viewer, offscreen rendering for demos	Lightweight 2D drawing; works headless
Matplotlib	Benchmark plots and figures	Publication-quality charts
Pillow	Demo GIF encoding	No external video tools needed
PyYAML + dataclasses	Typed configuration (configs/default.yaml)	Every parameter in one validated place
pytest	189 unit and end-to-end tests	Every phase verified before it was committed
ReportLab	This document	PDF generated from code, so it stays in sync
Git + GitHub	Version control, development branch per phase	Reviewable history, one commit per phase

Hardware constraint: an Intel i5-8365U laptop (4 cores, 16 GB RAM, integrated graphics, no CUDA GPU, limited disk). This ruled out Unity and GPU training and pushed the design toward efficient, vectorised CPU code.

5. System architecture

Every simulation step (dt = 0.05 s) runs the same pipeline; each stage only uses information a real drone could have.

Stage	What happens	Module
1. Sense	Neighbours within 15 m (KD-tree); 36-beam lidar; nearest obstacle points	sim/neighbors.py, agents/sensors.py
2. Decide (mission)	Rescue state machine, formation slots, terrain routes	mission/, formation/, terrain/
3. Plan	Path on the drone's own map; steering waypoint by string pulling	planning/
4. Control	Boids / formation / mission / learned policy → desired acceleration	control/, rl/
5. Safety	ORCA turns it into a collision-free velocity	control/orca.py
6. Physics	Integrate with speed/acceleration limits; walls; obstacle contact	sim/dynamics.py, sim/world.py
7. Map	Fold the lidar scan into the drone's own occupancy grid	mapping/occupancy.py
8. Communicate	Broadcast beacons, map cells, mission gossip; loss and latency	comms/network.py
9. Measure	Collisions, clearance, coverage, arrivals, formation error, ...	metrics.py

One controller interface. Every behaviour implements `compute(state, context) → acceleration`, so a hand-written controller, a learned policy, or either wrapped in ORCA can be swapped with one setting and benchmarked under identical conditions.



The live viewer during a search-and-rescue mission (130 s): the team's shared map (green), radio links, victims (×), the base station (blue square) and live statistics.

6. Key results

All numbers come from the benchmark scripts in `experiments/`, repeated over several random seeds. The full tables are in the project README.

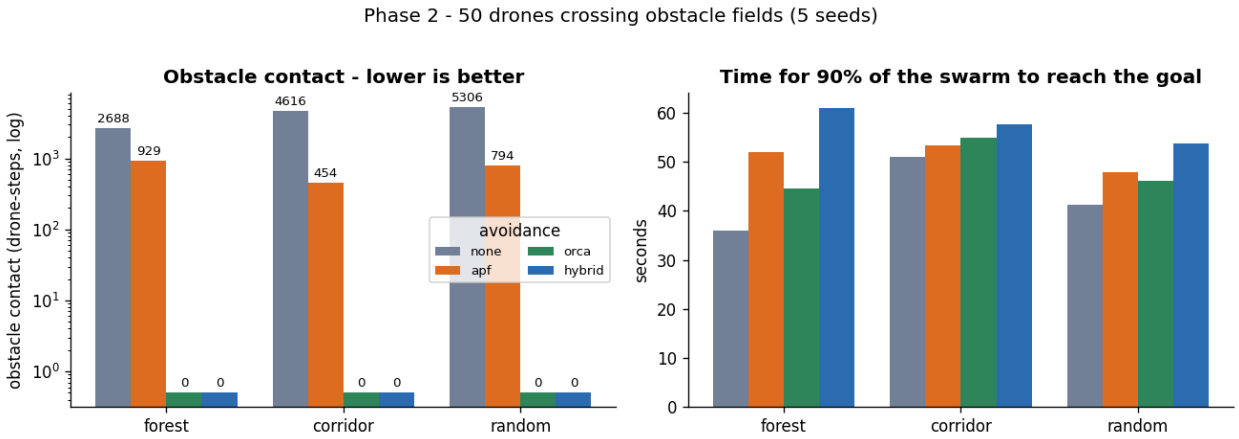


Figure 1 - Only ORCA keeps every drone off every obstacle (50 drones, 3 obstacle fields, 5 seeds).

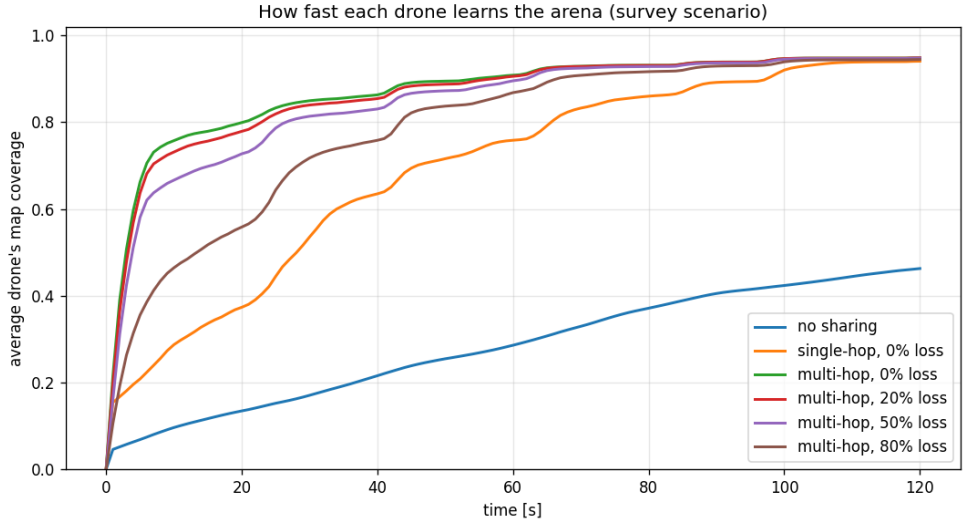


Figure 2 - Multi-hop map sharing: the average drone knows 85% of the arena after 30 s, against 17% without sharing; robust to heavy packet loss.

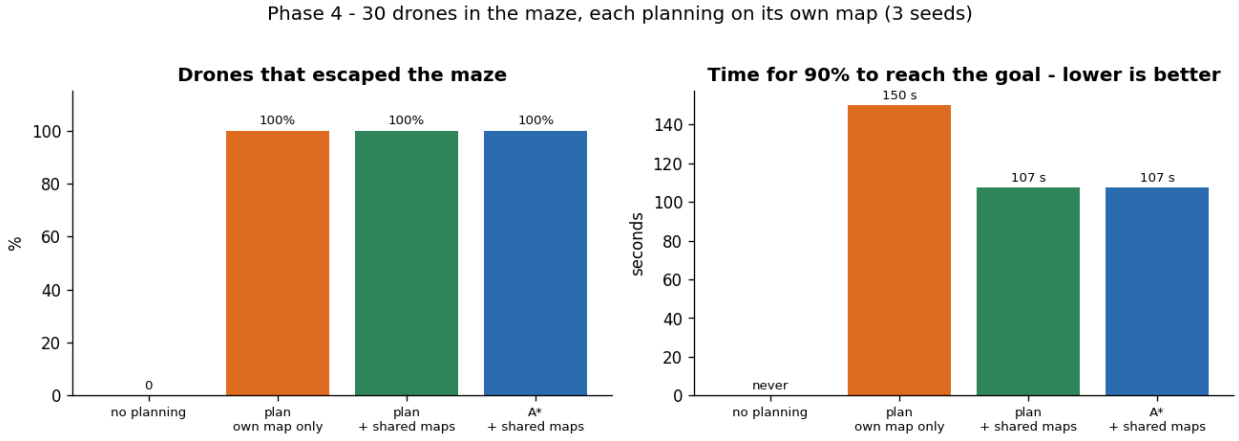


Figure 3 - In a maze, reactive flight never escapes; planning on each drone's own map always does, and shared maps make it 29% faster.

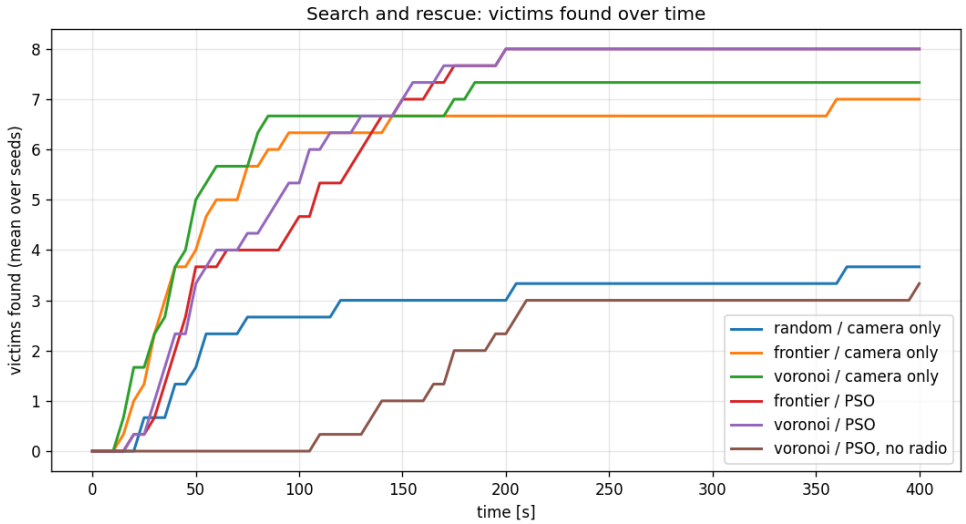


Figure 4 - Search and rescue: only the signal-guided PSO finds all 8 victims in every run; without radio the swarm finds 3.3.

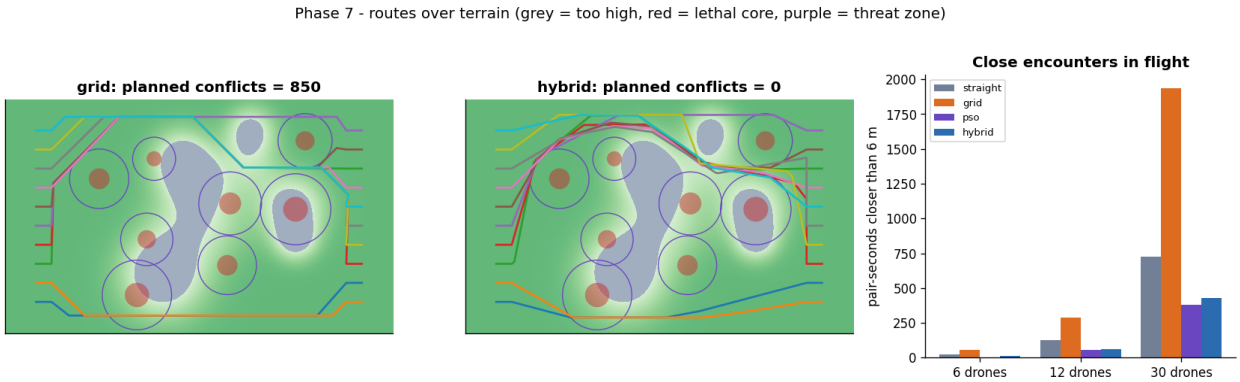


Figure 5 - Planning in space and time: hybrid PSO routes have no planned conflicts and 4–5× fewer close encounters than grid routes.

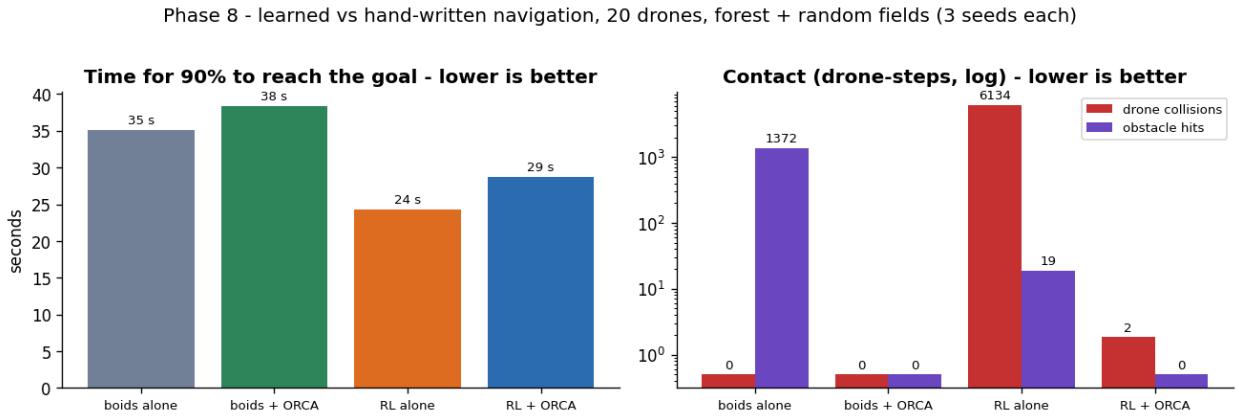


Figure 6 - Learned navigation is fast but unsafe alone; wrapped in ORCA it is ~25% faster than the hand-written stack with almost no contact.

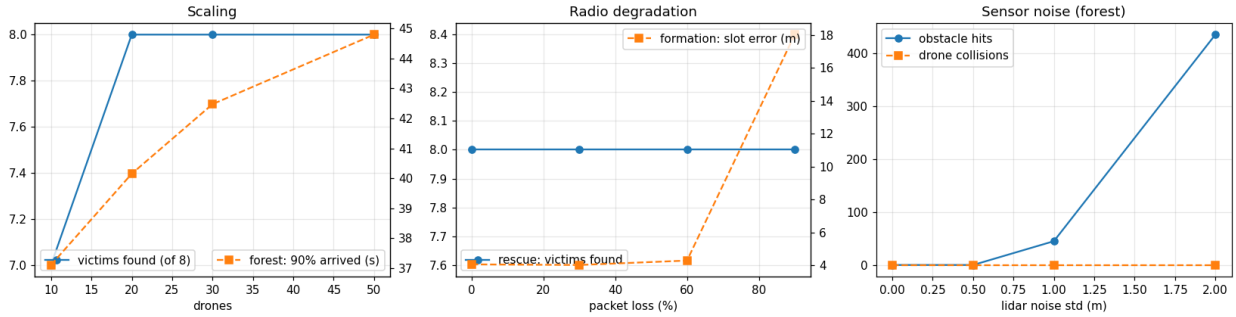


Figure 7 - Graceful degradation under scaling, radio loss and sensor noise.

7. Lessons learned

Many of the most useful lessons came from bugs that tests and benchmarks exposed. These are the ones worth remembering for any robotics or multi-agent project.

Research lessons

- **Local rules really do produce global behaviour**, but only if safety has priority over everything else. Prioritised acceleration and ORCA turned an approximate flock into a provably collision-free one.
- **Communication multiplies capability.** Map sharing gave each drone 5× more knowledge; switching the radio off reduced rescue success by 60%. Robustness comes from redundancy (gossip, anti-entropy, relays).
- **Plan with what you know, assume optimistically, replan fast.** Optimistic planning on each drone's own map solved a maze none of them had seen.
- **Combine methods instead of choosing one.** PSO + graph search (hybrid), learned policy + ORCA, frontier exploration + signal search: every best result was a combination.
- **Report negative results.** Learned policies did not beat hand-written control at 20 drones. That is a real finding about data and compute budgets, not a failure to hide.
- **Wrong information is worse than missing information.** Losing 60% of lidar beams was harmless; 1 m of noise caused crashes.

Engineering lessons

- **Measure, don't assume.** Profiling found that ray casting, graph construction and path extraction dominated; vectorisation made them 3–6× faster with identical results (checked against brute force).
- **Test the invariants, not just the outputs.** Tests caught a ray-casting bug that made drones “see” a wall inside a gap, path extraction that cut corners, and a route score that depended on its batch.
- **Metrics can lie.** A stuck detector fired ~200 false alarms per run; “left behind” counted a correct column as stranded. Every metric needs a sanity check against what actually happened.
- **In RL, an unobservable penalty makes learning go backwards**, and one shared gradient clip can silently stop the policy from learning. Verify the learner on a toy problem first.
- **Reproducibility needs separate random streams**: radio randomness must not change how drones fly, so experiments differ only in what they are meant to test.

8. From simulation to the real world

A simulation is valuable only if its ideas survive contact with reality. This section lists what the simulation assumes, what real systems use instead, a step-by-step path to real drones, and the real-world problems the project's methods address.

8.1 The reality gap

Simulation assumes	Reality	How to close the gap
2D world, point-mass drones	3D flight, quadrotor dynamics (thrust, attitude, drag, rotor lag)	3D extension; low-level flight controller (PX4 / ArduPilot) tracks the velocity commands
Perfect self-position	GPS ±2–5 m (RTK ±2 cm); no GPS indoors; drift	RTK-GPS outdoors; UWB, motion capture or visual-inertial odometry indoors; filter estimates
Ideal lidar, Gaussian noise	Reflections, dust, sunlight, weight and power limits	Sensor fusion (lidar + depth camera); noise-aware mapping; the Phase 9 tests show where it breaks
Simple radio (range, loss, latency)	Interference, obstacles, bandwidth, security	Mesh networking (Wi-Fi 802.11s, ESP-NOW, LoRa for long range); encryption; keep messages small
No wind or weather	Gusts, rain, temperature	Robust control margins; ORCA with larger safety margins
Linear battery model	Voltage sag, temperature, payload effects	Measured discharge curves; bigger reserve
Instant computation	Embedded CPUs, real-time limits	Profile on target hardware (e.g. Jetson, Raspberry Pi); the vectorised design helps
Learned policies trained in sim	Sim-to-real gap for RL	Domain randomisation, system identification, always keep a verified safety layer (ORCA)

8.2 A step-by-step path to real drones

Step	What to do	Tools
1. Higher-fidelity simulation	Port the controllers to a 3D physics simulator with real quadrotor models	Gazebo, PX4 SITL, gym-pybullet-drones, NVIDIA Isaac Sim
2. Robot middleware	Wrap each module (sensing, mapping, planning, ORCA, comms) as a ROS 2 node	ROS 2, rclpy, Nav2 (for ideas)
3. Flight-controller integration	Send velocity setpoints in offboard mode; let the autopilot stabilise	PX4 / ArduPilot, MAVLink, MAVSDK, MAVROS
4. Small indoor swarm	Test with tiny drones in a safe, netted room with external positioning	Bitcraze Crazyflie + Crazyswarm2, Lighthouse / UWB / motion capture
5. Real sensing and comms	Add onboard range sensors and a mesh radio; tune the noise models from data	Multi-ranger / ToF decks, small lidars, ESP-NOW / 802.11s mesh
6. Outdoor field tests	Few drones, geofenced, with pilot override, then scale up	RTK-GPS, telemetry radios, QGroundControl
7. Validation	Repeat the project's benchmarks on hardware; compare to simulation	Same metrics: collisions, clearance, coverage, arrival time

8.3 Mapping each module to its real-world counterpart

Project module	Real-world counterpart
Boids + ORCA (control/)	Onboard reactive collision avoidance; ORCA/RVO are used on real robots and drone swarms (e.g. RVO2 library)
Lidar ray casting (agents/sensors.py)	2D/3D lidar, time-of-flight sensors, depth cameras

Project module	Real-world counterpart
Occupancy grids (mapping/)	OctoMap / 2D costmaps in ROS; SLAM systems for localisation + mapping
Radio, gossip, beacons (comms/)	Mesh networking, DDS (the ROS 2 transport), Remote ID broadcasts
Planner (planning/)	Global planners in navigation stacks; replanning on new map data
Formations (formation/)	Drone light shows, cooperative transport, convoy escort
Rescue mission (mission/)	Multi-UAV search missions with thermal cameras, phone-signal detection
Route planning (terrain/)	Pre-flight mission planning with airspace, terrain and no-fly constraints
Learned policies (rl/)	Learned controllers deployed with safety filters; sim-to-real with domain randomisation
Fault injection (faults)	Failure-mode testing, redundancy design, return-to-home and failsafes

8.4 Real-world applications

Application	How the project's methods apply	What else is needed
Disaster search and rescue (earthquakes, floods, landslides)	Frontier + Voronoi exploration, signal-guided PSO (phones), battery-aware returns, relaying reports to a base	Thermal / RGB cameras, person detection, weather-proof drones, coordination with rescue teams
Wildfire and environmental monitoring	Coverage, threat-zone route planning (fire = threat), shared maps	Thermal sensing, long-range radio, heat-tolerant hardware
Precision agriculture	Coverage planning, formations for spraying lanes, shared crop maps	Multispectral cameras, payload management, regulation for spraying
Infrastructure inspection (bridges, power lines, wind turbines)	Path planning near structures, ORCA close to obstacles, multi-drone task allocation	High-precision positioning, detailed 3D models
Emergency communication relay	Multi-hop radio, connectivity, formations that keep links alive	Communications payloads, endurance (tethered or long-flight drones)
Warehouse inventory and logistics	Indoor navigation, collision avoidance in dense traffic, task allocation	Indoor positioning, barcode / RFID scanning
Drone light shows	Formation shapes, Hungarian slot assignment, time-deconflicted trajectories	Precise RTK positioning, choreography tools
Delivery and urban air mobility	Time-deconflicted route planning, threat (no-fly) avoidance	Airspace integration (UTM), certification, Remote ID

8.5 Safety, legal and ethical considerations

- **Regulation.** Drone operations are governed by national aviation authorities, for example the Civil Aviation Authority of Sri Lanka, the FAA (USA, Part 107) and EASA (EU). Swarms usually need special permissions: beyond-visual-line-of-sight flight, multiple aircraft per pilot, and flight over people.
- **Safety by design.** Geofences, return-to-home on low battery or lost link, a human override, and a verified safety layer (here: ORCA) around any learned or experimental controller.
- **Privacy.** Cameras and phone-signal detection can collect personal data; search missions need clear purpose, data minimisation and deletion policies.
- **Dual use.** Swarm technology can be misused. The project focuses on humanitarian and civilian applications; responsible publication and adherence to law are part of the engineering.
- **Reliability honesty.** Report limitations openly (e.g. large formations, learned policies at scale) so nobody deploys a method beyond what has been demonstrated.

9. Future research directions

- **3D swarm**: altitude as a real dimension, 3D ORCA, 3D occupancy (OctoMap-style) and terrain-following flight.
- **Incremental replanning**: full D* Lite instead of recomputing the cost field; any-angle paths (Theta*).
- **Distributed SLAM**: drones that estimate their own positions and merge maps without perfect localisation.
- **Better multi-agent learning**: graph neural network policies that work for any number of neighbours, longer training on a GPU (Colab / Kaggle), domain randomisation for sim-to-real.
- **Large formations**: hierarchical formations (squads with sub-leaders) for 50+ drones.
- **Consensus-based leader election and task allocation** that tolerate message loss and network partitions.
- **Charging logistics**: queueing at charging pads, battery swapping, mobile base stations.
- **Adversarial and security aspects**: spoofed messages, jamming, Byzantine-robust gossip.
- **Hardware validation** with a small Crazyflie swarm, repeating the project's benchmarks.

Glossary

Term	Meaning
Agent	An autonomous decision maker; here, one drone
Emergence	Global behaviour arising from local interactions without central control
Polarisation	How aligned the swarm's headings are (0 random, 1 identical)
Velocity obstacle	Set of velocities that cause a collision within a time horizon
Occupancy grid	Map of cells each holding the probability of being occupied
Log-odds	log(p / (1 − p)); turns Bayesian updates into additions
Frontier	Boundary between known-free and unknown space
Voronoi partition	Division of space by which robot is nearest
Admissible heuristic	Never overestimates the remaining cost, so A* stays optimal
Gossip / anti-entropy	Spreading data by neighbour exchanges / periodically repairing missed updates
PSO	Particle Swarm Optimisation: population search guided by personal and global bests
PPO / GAE	Clipped policy-gradient RL algorithm / advantage estimator
CTDE	Centralised training, decentralised execution
Sim-to-real gap	Differences that make simulated results fail on real hardware
Graceful degradation	Performance that drops gradually, not catastrophically, under faults

10. References

Key papers and books behind the methods used or studied in the project, grouped roughly in the order the topics appear: swarms, avoidance, mapping, communication, planning, formations, optimisation, missions, learning and real-world platforms.

[1] C. W. Reynolds, “Flocks, herds and schools: A distributed behavioral model,” *Computer Graphics (SIGGRAPH)*, 21(4), pp. 25–34, 1987.

[2] C. W. Reynolds, “Steering behaviors for autonomous characters,” *Game Developers Conference*, 1999.

[3] T. Vicsek et al., “Novel type of phase transition in a system of self-driven particles,” *Physical Review Letters*, 75(6), p. 1226, 1995.

[4] R. Olfati-Saber, “Flocking for multi-agent dynamic systems: Algorithms and theory,” *IEEE Trans. Automatic Control*, 51(3), pp. 401–420, 2006.

[5] M. Brambilla, E. Ferrante, M. Birattari, M. Dorigo, “Swarm robotics: A review from the swarm engineering perspective,” *Swarm Intelligence*, 7(1), pp. 1–41, 2013.

[6] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen, V. Kumar, “A survey on aerial swarm robotics,” *IEEE Trans. Robotics*, 34(4), pp. 837–855, 2018.

[7] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *Int. J. Robotics Research*, 5(1), pp. 90–98, 1986.

[8] P. Fiorini, Z. Shiller, “Motion planning in dynamic environments using velocity obstacles,” *Int. J. Robotics Research*, 17(7), pp. 760–772, 1998.

[9] J. van den Berg, M. Lin, D. Manocha, “Reciprocal velocity obstacles for real-time multi-agent navigation,” *IEEE ICRA*, 2008.

[10] J. van den Berg, S. J. Guy, M. Lin, D. Manocha, “Reciprocal n-body collision avoidance,” *Robotics Research (ISRR)*, Springer STAR vol. 70, pp. 3–19, 2011.

[11] A. D. Ames et al., “Control barrier functions: Theory and applications,” *European Control Conference*, 2019.

[12] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, 22(6), pp. 46–57, 1989.

[13] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005.

[14] A. Demers et al., “Epidemic algorithms for replicated database maintenance,” *ACM PODC*, 1987.

[15] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, 1, pp. 269–271, 1959.

[16] P. E. Hart, N. J. Nilsson, B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, 4(2), pp. 100–107, 1968.

[17] S. Koenig, M. Likhachev, “D* Lite,” *AAAI Conference on Artificial Intelligence*, 2002.

[18] A. Nash, K. Daniel, S. Koenig, A. Felner, “Theta*: Any-angle path planning on grids,” *AAAI*, 2007.

[19] J. P. Desai, J. P. Ostrowski, V. Kumar, “Modeling and control of formations of nonholonomic mobile robots,” *IEEE Trans. Robotics and Automation*, 17(6), pp. 905–908, 2001.

[20] W. Ren, R. W. Beard, *Distributed Consensus in Multi-vehicle Cooperative Control*, Springer, 2008.

[21] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, 2, pp. 83–97, 1955.

[22] J. Kennedy, R. Eberhart, “Particle swarm optimization,” *IEEE Int. Conf. Neural Networks*, 1995.

[23] Y. Shi, R. Eberhart, “A modified particle swarm optimizer,” *IEEE Int. Conf. Evolutionary Computation*, 1998.

[24] M. Erdmann, T. Lozano-Pérez, “On multiple moving objects,” *Algorithmica*, 2, pp. 477–521, 1987.

[25] D. H. Douglas, T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica*, 10(2), pp. 112–122, 1973.

[26] B. Yamauchi, “A frontier-based approach for autonomous exploration,” *IEEE CIRA*, 1997.

[27] J. Cortés, S. Martínez, T. Karataş, F. Bullo, “Coverage control for mobile sensing networks,” *IEEE Trans. Robotics and Automation*, 20(2), pp. 243–255, 2004.

[28] H.-L. Choi, L. Brunet, J. P. How, “Consensus-based decentralized auctions for robust task allocation,” *IEEE Trans. Robotics*, 25(4), pp. 912–926, 2009.

[29] H. Garcia-Molina, “Elections in a distributed computing system,” *IEEE Trans. Computers*, C-31(1), pp. 48–59, 1982.

[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.

[31] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *ICLR*, 2016.

[32] R. Lowe et al., “Multi-agent actor-critic for mixed cooperative-competitive environments,” *NeurIPS*, 2017.

[33] C. Yu et al., “The surprising effectiveness of PPO in cooperative multi-agent games,” *NeurIPS Datasets and Benchmarks*, 2022.

[34] S. Huang et al., “The 37 implementation details of proximal policy optimization,” *ICLR Blog Track*, 2022.

[35] J. Tobin et al., “Domain randomization for transferring deep neural networks from simulation to the real world,” *IEEE/RSJ IROS*, 2017.

[36] J. A. Preiss, W. Hönig, G. S. Sukhatme, N. Ayanian, “Crazyswarm: A large nano-quadcopter swarm,” *IEEE ICRA*, 2017.

[37] W. Hönig, J. A. Preiss, T. K. S. Kumar, G. S. Sukhatme, N. Ayanian, “Trajectory planning for quadrotor swarms,” *IEEE Trans. Robotics*, 34(4), pp. 856–869, 2018.

[38] J. Panerati et al., “Learning to fly: A gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control,” *IEEE/RSJ IROS*, 2021.

[39] L. Meier, D. Honegger, M. Pollefeys, “PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms,” *IEEE ICRA*, 2015.

[40] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, 7(66), 2022.

[41] S. Shah, D. Dey, C. Lovett, A. Kapoor, “AirSim: High-fidelity visual and physical simulation for autonomous vehicles,” *Field and Service Robotics*, Springer, 2018.

Project source code, benchmarks and demos:
github.com/Thilina-Samarasinghe/autonomous-drone-swarm. This document is generated by `python -m experiments.make_research_guide`.